Reliability cheat sheet

MC 451 / MC 501 · check your own coding: pass 1 against pass 2

This sheet covers the commands that check how consistent your coding is. You code the same items twice, a few days apart, with your first codes hidden the second time. Then R compares pass 1 with pass 2. Each entry says what the command does in one line, shows it running on a real coded pilot, and shows the exact output R prints. **On your data** rewrites the call with plain column names you can copy, and **If it breaks** names the error you are most likely to see and what it means.

`kappa2()` and `kripp.alpha()` are from **irr**, which the course package installs for you; the code names them in full (`irr::kappa2()`), so you never load irr yourself. `reliability()` is from **v2v**. Everything else is **dplyr** and **tidyr**.

Stability: how closely you agree with yourself when you code the same items twice. You are the only coder in this course, so pass 1 and pass 2 are both yours. In your write-up, call the number a stability figure, never agreement between coders.

Every reliability block starts with these lines, so the examples below assume them:

```
library(tidyverse)    # dplyr, readr, and tidyr
library(v2v)           # the course package, for reliability()
options(scipen = 10)  # long numbers print in full
```

The data these examples use

The tone codes from the MC 451 worked example's pilot: 20 tweets about playing in VR, each coded 1, 2, or 3 for tone toward VR, once on October 31 and again on November 3, 72 hours later. The same codes are in the coded pilot data (CSV) as `tone_p1` and `tone_p2`.

The R Companion's **Reliability grid** saves your two passes for one variable as `data/derived/reliability-<variable>.csv`, with the columns `pass_1`, `pass_2`, `pass_1_date`, and `pass_2_date`, then writes the code below into your script. Every example on this sheet is a line the grid writes, run on the tone file.

`read_csv()`: read the grid's saved file

Reads the file the grid saved and names it after the variable, here `tone_passes`. It prints a short note about the column types it guessed; that note is not an error.

```
tone_passes <- read_csv("data/derived/reliability-tone.csv")
tone_passes
```

```
Rows: 20 Columns: 4
──                                                                      Column
specification ──────────────────────────────────────────────────────────────
Delimiter: ","
dbl  (2): pass_1, pass_2
date (2): pass_1_date, pass_2_date

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
# A tibble: 20 × 4
   pass_1 pass_2 pass_1_date pass_2_date
    <dbl>  <dbl> <date>      <date>
 1      2      2 2026-10-31  2026-11-03
 2      1      1 2026-10-31  2026-11-03
 3      3      2 2026-10-31  2026-11-03
 4      3      3 2026-10-31  2026-11-03
 5      2      2 2026-10-31  2026-11-03
 6      2      2 2026-10-31  2026-11-03
 7      2      2 2026-10-31  2026-11-03
 8      2      2 2026-10-31  2026-11-03
 9      3      3 2026-10-31  2026-11-03
10      3      3 2026-10-31  2026-11-03
11      2      2 2026-10-31  2026-11-03
12      2      2 2026-10-31  2026-11-03
13      2      2 2026-10-31  2026-11-03
14      2      2 2026-10-31  2026-11-03
15      2      2 2026-10-31  2026-11-03
16      1      1 2026-10-31  2026-11-03
17      2      2 2026-10-31  2026-11-03
18      2      2 2026-10-31  2026-11-03
19      3      3 2026-10-31  2026-11-03
20      2      2 2026-10-31  2026-11-03
```

On your data: `your_passes <- read_csv("data/derived/reliability-your-variable.csv")`. If you coded in a coding sheet instead, your two columns are named like `tone_p1` and `tone_p2`; put those names wherever this sheet says `pass_1` and `pass_2`.

If it breaks: `does not exist in current working directory` means R is looking in a different folder. Open your workspace's top folder in VS Code, so `data/derived/` sits inside it.

Percent agreement: `mean(pass_1 == pass_2)`

The share of items where pass 1 and pass 2 gave the same code. `pass_1 == pass_2` is TRUE for each item that matches, and `mean()` turns those TRUEs into a share between 0 and 1.

```
tone_passes %>%
  summarize(
    items = n(),
    share_agree = mean(pass_1 == pass_2)
  )
```

```
# A tibble: 1 × 2
  items share_agree
  <int>       <dbl>
1    20        0.95
```

The two passes matched on 19 of 20 tweets, 95 percent.

On your data: `your_passes %>% summarize(items = n(), share_agree = mean(pass_1 == pass_2))`

If it breaks: `share_agree` prints `NA` when any item is missing a code in one pass. Code the blank item, or drop it first with `filter(!is.na(pass_1), !is.na(pass_2))`, as the grid does for a coding sheet.

`count()`: every pair of codes

Counts how many items got each pair of codes. Rows where the two columns match are agreements; any other row is a disagreement. The `n` column adds up to the number of items.

```
tone_passes %>%
  count(pass_1, pass_2)
```

```
# A tibble: 4 × 3
  pass_1 pass_2     n
   <dbl>  <dbl> <int>
1      1      1     2
2      2      2    13
3      3      2     1
4      3      3     4
```

One tweet was a 3 in pass 1 and a 2 in pass 2. Every other row is a match.

On your data: `your_passes %>% count(pass_1, pass_2)`

If it breaks: a row with `NA` in it is an item you coded only once. It counts as neither an agreement nor a disagreement until you code it.

`filter(pass_1 != pass_2)`: find the disagreements

Keeps only the items the two passes coded differently. `!=` means "is not equal to", and `row_number()` adds each item's place in the file, so you can find it in your coding sheet. Each disagreement points at a rule in your codebook that is unclear, so these are the items your edge cases and revisions come from.

```
tone_passes %>%
  mutate(item = row_number()) %>%
  filter(pass_1 != pass_2)
```

```
# A tibble: 1 × 5
  pass_1 pass_2 pass_1_date pass_2_date  item
   <dbl>  <dbl> <date>      <date>      <int>
1      3      2 2026-10-31  2026-11-03      3
```

On your data: `your_passes %>% mutate(item = row_number()) %>% filter(pass_1 != pass_2)`

If it breaks: an empty table (`0 × 5`) means the passes never disagreed. That is good news, not a failure.

`irr::kappa2()`: Cohen's kappa

Cohen's kappa is agreement after taking out the matches you would get by luck alone. 1 means the passes always matched, and 0 means they matched no more often than chance. Give `kappa2()` a table with exactly two columns: pass 1 and pass 2.

```
tone_passes %>%
  select(pass_1, pass_2) %>%
  irr::kappa2()
```

```
 Cohen's Kappa for 2 Raters (Weights: unweighted)

 Subjects = 20
   Raters = 2
    Kappa = 0.897

        z = 5.17
  p-value = 0.000000232
```

The number to report is `Kappa = 0.897`. "Subjects" is the items, and "Raters = 2" is your two passes. Leave the z and the p-value out of your write-up; they test whether kappa is above zero, which is not the question a stability check asks.

On your data: `your_passes %>% select(pass_1, pass_2) %>% irr::kappa2()`

If it breaks: `Number of raters exeeds 2` means the table had more than two columns (here, the two date columns came along). Add `select(pass_1, pass_2)` before `kappa2()`. And `Kappa = NaN` means both passes used one code for every item, so kappa cannot be computed. Say so in your write-up instead of reporting a number.

Number the codes for alpha

`kripp.alpha()` wants numbers, so the grid first makes one list of every code either pass used. `pivot_longer()` stacks the two columns into one called `code`, `distinct()` keeps each code once, and `pull()` takes the list out of the table.

```
codes <- tone_passes %>%
  pivot_longer(c(pass_1, pass_2), values_to = "code") %>%
  distinct(code) %>%
  pull(code)
codes
```

```
[1] 2 1 3
```

The list is in the order the codes first appear. Alpha only checks whether two codes are the same, so the order does not change the result.

On your data: the same lines with `your_passes` in place of `tone_passes`. Codes written as words, such as `"calm"`, work too.

If it breaks: `object 'codes' not found` in the next step means this step did not run. Run it first, then the alpha step.

`t()`: one row per pass

`kripp.alpha()` wants the passes as **rows** and the items as **columns**, the opposite of your table. `factor(.x, levels = codes)` turns each code into its place in the list (2 becomes 1, 1 becomes 2, 3 stays 3), and `t()` turns the table on its side.

```
tone_passes %>%
 mutate(across(c(pass_1, pass_2), ~ as.integer(factor(.x, levels = codes)))) %>%
  select(pass_1, pass_2) %>%
  t()
```

```
       [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10] [,11] [,12] [,13]
pass_1    1    2    3    3    1    1    1    1    3     3     1     1     1
pass_2    1    2    1    3    1    1    1    1    3     3     1     1     1
       [,14] [,15] [,16] [,17] [,18] [,19] [,20]
```

```
pass_1    1    1    2    1    1    3    1
pass_2    1    1    2    1    1    3    1
```

Two rows, one per pass, and 20 columns, one per tweet. Column 3 is the one disagreement.

On your data: the same lines with `your_passes` in place of `tone_passes`.

If it breaks: this step only reshapes the table; the next step is where a wrong shape shows up.

`irr::kripp.alpha()`: Krippendorff's alpha

Krippendorff's alpha is a reliability score like kappa that also works with ranked or numeric codes, more than two passes, and a missing code. With two passes and named categories, it lands very close to kappa. `method = "nominal"` says the codes are names with no order.

```
tone_passes %>%
 mutate(across(c(pass_1, pass_2), ~ as.integer(factor(.x, levels = codes)))) %>%
  select(pass_1, pass_2) %>%
  t() %>%
  irr::kripp.alpha(method = "nominal")
```

```
 Krippendorff's alpha

 Subjects = 20
   Raters = 2
    alpha = 0.899
```

The number to report is `alpha = 0.899`. The worked example rounds it to .90.

On your data: the same lines with `your_passes` in place of `tone_passes`.

If it breaks: `cannot xtfrm data frames` means the table reached `kripp.alpha()` without going through `t()`. Put the `t() %>%` line back above it.

`reliability()`: the check, in one call

From **v2v**. Takes the pass 1 column and the pass 2 column and prints the items coded, the percent agreement, and kappa or alpha. The grid writes it last, as a check that the irr lines above gave the right number. `pull()` takes one named column out of the table.

```
reliability(
  pull(tone_passes, pass_1),
  pull(tone_passes, pass_2),
  method = "kappa"
)
```

```
Reliability: Cohen's kappa

  Items coded          20
  Observed agreement   0.95
  Cohen's kappa        0.897  (almost perfect (Landis & Koch 1977))

  The same number, without v2v: irr::kappa2(data.frame(pass_1, pass_2))
```

With `method = "alpha"` it prints Krippendorff's alpha instead:

```
reliability(
  pull(tone_passes, pass_1),
  pull(tone_passes, pass_2),
  method = "alpha"
)
```

```
Reliability: Krippendorff's alpha

  Items coded          20
  Observed agreement   0.95
  Krippendorff's alpha 0.899  (almost perfect (Landis & Koch 1977))

  The same number, without v2v: irr::kripp.alpha(rbind(pass_1, pass_2), method
= "nominal"), with the codes numbered
```

"Observed agreement" here is the same 0.95 as `share_agree` above.

On your data: `reliability(pull(your_passes, pass_1), pull(your_passes, pass_2), method = "kappa")`

If it breaks: `` `coder_a` and `coder_b` must be the same length `` means the two columns came from different tables, or one was cut short. Pull both from the same table.

Read the number against the threshold

Name your threshold before you code, then compare the number with it. These are the floors the course decks use:

Statistic	Good enough to go on	What to do below it
Cohen's kappa	0.70 or higher is the course floor; 0.80 is comfortable	Below 0.70: fix the rule and pilot again
Krippendorff's alpha	0.80 or higher	0.667 to 0.799: report it and call the results that rest on it

Statistic	Good enough to go on	What to do below it
		tentative; below 0.667, fix the rule and pilot again
Percent agreement	Never on its own	Report it beside kappa or alpha, never in place of one

The tone example clears both floors: kappa 0.897 and alpha 0.899.

- **The label in brackets is not the threshold.** `reliability()` prints a description from Landis and Koch (1977), such as "substantial", for both kappa and alpha. Compare the number with the floor in the table, not the label.
- **Why percent agreement is not enough:** with few codes, chance alone produces a lot of matches. Kappa and alpha take those lucky matches out, which is why the rubric asks for one of them.
- **One number per variable.** Run the block for every variable you coded. The worked example reports four: learning 1.00, tone .90, affordance .91, and constraint .86.

Two data sources: one number each

If your study codes two sources, such as Reddit posts and news articles, keep one coding sheet per source, each with its own pass 1 and pass 2. Run the block once on each source's file and report both numbers in your write-up, one per source.

On your data: with two grid files, run every step above on the first, then again on the second:

```
reddit_passes <- read_csv("data/derived/reliability-reddit_tone.csv")
news_passes <- read_csv("data/derived/reliability-news_tone.csv")
```

If it breaks: the grid names its file after the variable, so a second grid run for the same variable name saves over the first. Give each source's variable its own name in the grid, such as `reddit_tone` and `news_tone`.